

Caso Practico Sockets UDP y TCP

Contenido

Caso Practico Sockets Sockets.....	1
Introduction.....	2
Objetivos conceptuales	2
Objetivos técnicos	2
Introducción a la Interfaz Sockets.....	3
Concepto general.....	3
Tipos principales de sockets:.....	3
Caso Práctico: Sistema de Monitoreo Industrial Distribuido para Telemetría y Alertas en Tiempo Real	4
Ejercicio a resolver	4
Arquitectura de la aplicación.....	5
Formato de mensajes.....	5
UDP:	5
TCP:.....	5
Dashboard de monitoreo (modo consola o GUI opcional)	6
Guía ejemplos de código.....	6
Sockets en Python	6
Aplicaciones UDP.....	6
Ejemplo de Servidor UDP en Python:	6
Ejemplo de Cliente UDP en Python:	6
Aplicaciones TCP.....	7
Ejemplo de Servidor TCP en Python:.....	7
Ejemplo de Cliente TCP en Python:	7
Guía paso a paso en Python	7
Sockets en C#	8
Ejemplo de Servidor UDP en .NET Core (Program.cs)	8
Ejemplo de Cliente UDP en .NET Core (Program.cs).....	8
Aplicaciones TCP en .NET Core (C#)	9
Ejemplo de Servidor TCP en .NET Core (Program.cs)	9
Ejemplo de Cliente TCP en .NET Core (Program.cs)	9
Sockets en NodeJS	10

Ejemplo de Servidor UDP:.....	10
Ejemplo de Cliente UDP:.....	10
Ejemplo de Servidor TCP:	10
Ejemplo de Cliente TCP:.....	11

Introduction

El presente trabajo práctico tiene como finalidad que el estudiante adquiera una comprensión profunda, aplicada y comparativa del funcionamiento de la comunicación entre procesos distribuidos mediante el uso de **sockets**, entendidos como la base fundamental de gran parte de las aplicaciones de red modernas.

A través del diseño, implementación, prueba y análisis de aplicaciones **cliente-servidor**, el alumno deberá comprender cómo dos o más procesos pueden intercambiar información a través de una red local o global utilizando distintos protocolos de transporte, principalmente **TCP (Transmission Control Protocol)** y **UDP (User Datagram Protocol)**.

Desde una perspectiva académica y profesional, este trabajo busca que el estudiante:

Objetivos conceptuales

- Comprenda qué es la comunicación entre procesos (Inter Process Communication – IPC) en entornos de red.
- Analice el modelo cliente-servidor como arquitectura básica de comunicación distribuida.
- Diferencie las funciones de las capas del modelo TCP/IP involucradas en la comunicación.
- Entienda el rol de las direcciones IP, puertos, sockets y protocolos de transporte.
- Distinga las características, ventajas, limitaciones y casos de uso de TCP y UDP.
- Relacione el uso de sockets con aplicaciones reales.

Objetivos técnicos

- Implementar aplicaciones funcionales utilizando sockets en **Python**, **C#** y **NodeJS**, desarrollando competencias multiplataforma y comparativas.
- Construir clientes y servidores capaces de:
 - establecer conexiones;
 - enviar y recibir datos;
 - manejar errores;
 - cerrar sesiones correctamente;
 - trabajar en entornos locales y en red.
- Implementar soluciones tanto:
 - **orientadas a conexión (TCP)**, priorizando confiabilidad;
 - **no orientadas a conexión (UDP)**, priorizando velocidad y simplicidad.
- Analizar diferencias sintácticas, estructurales y de rendimiento entre distintos lenguajes de programación para un mismo problema de red.

En términos generales, este trabajo no busca únicamente “hacer funcionar” programas de red, sino formar al estudiante en la comprensión integral de **cómo se comunican sistemas reales**, cómo se diseñan soluciones robustas y qué decisiones técnicas deben tomarse según el contexto operativo. El estudio de sockets es clave porque constituye la base técnica de innumerables tecnologías modernas:

- Navegadores web
- Servidores HTTP
- Sistemas bancarios
- Videojuegos multijugador
- Aplicaciones móviles
- Microservicios
- Cloud Computing
- Sistemas distribuidos
- Ciberseguridad

Comprender sockets permite entender desde conexiones simples hasta arquitecturas empresariales complejas. Este trabajo permite comprender las diferencias prácticas entre TCP y UDP, su uso en distintas plataformas, y cómo cada lenguaje maneja la interfaz de sockets. Se recomienda realizar pruebas locales y luego extenderlas a una red LAN para observar las diferencias en latencia y confiabilidad.

Introducción a la Interfaz Sockets

La interfaz **Sockets** constituye uno de los mecanismos fundamentales de comunicación en redes de computadoras y representa una abstracción de software que permite a dos procesos, ejecutándose en una misma máquina o en sistemas remotos, intercambiar información de forma estructurada a través de protocolos de red.

Concepto general

Un **socket** puede definirse como un extremo o punto terminal de comunicación entre dos aplicaciones. Funciona como una interfaz entre la aplicación y la capa de transporte del modelo TCP/IP, permitiendo que los programas utilicen la red sin necesidad de gestionar directamente todos los detalles de bajo nivel.

En términos prácticos, un socket actúa de manera similar a un “enchufe lógico” mediante el cual una aplicación puede:

- abrir una conexión;
- escuchar solicitudes;
- enviar datos;
- recibir información;
- cerrar la comunicación.

Tipos principales de sockets:

Tipo	Características	Uso típico
TCP (SOCK_STREAM)	Confiable, orientado a conexión, garantiza el orden de los datos.	Aplicaciones que requieren confiabilidad (HTTP, FTP, SSH).
UDP (SOCK_DGRAM)	No orientado a conexión, sin garantía de entrega ni orden.	Aplicaciones en tiempo real (VoIP, streaming, juegos).

Caso Práctico: Sistema de Monitoreo Industrial Distribuido para Telemetría y Alertas en Tiempo Real

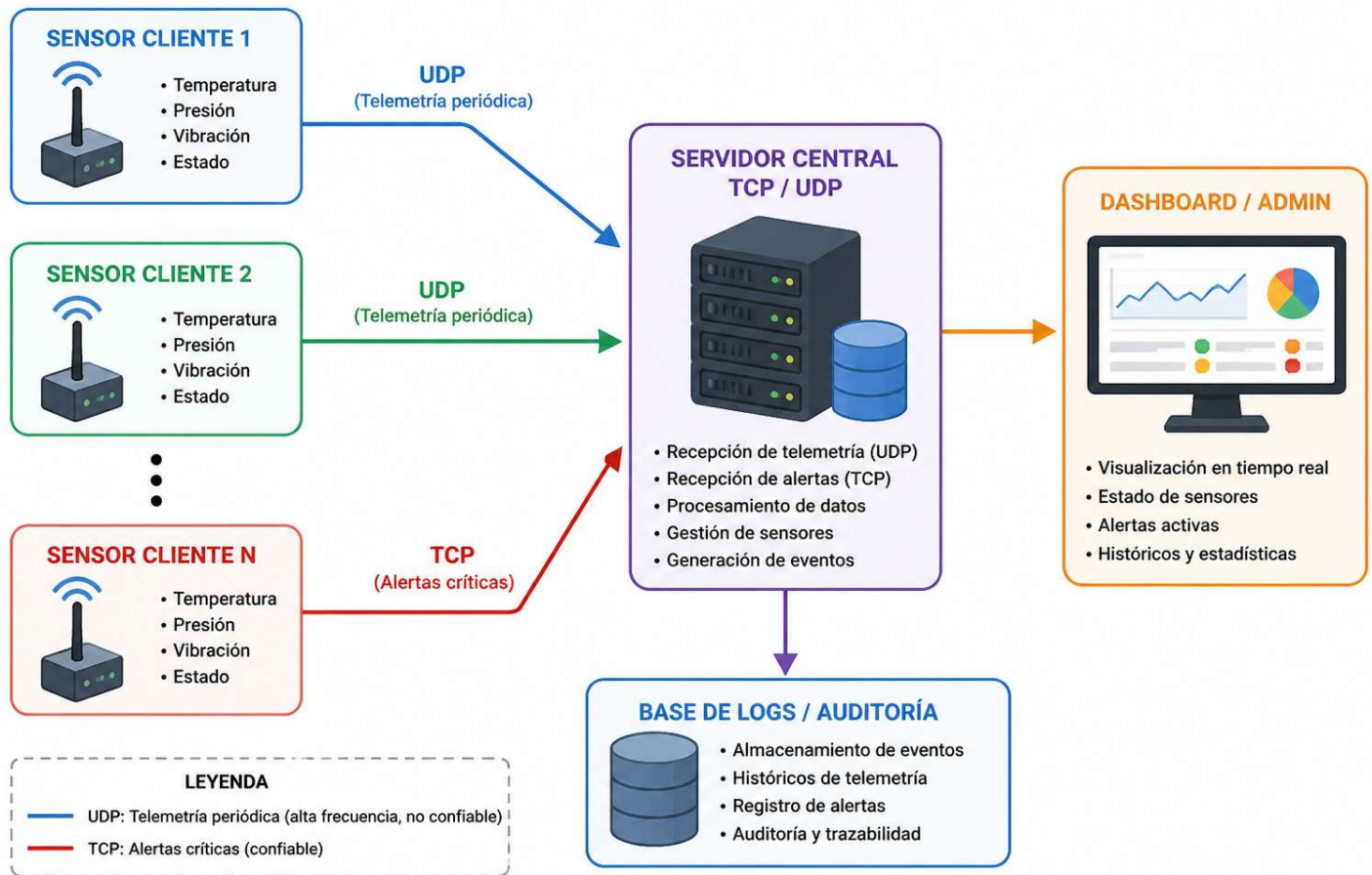
Una empresa industrial dedicada a procesos de manufactura avanzada necesita implementar un sistema de monitoreo distribuido para supervisar en tiempo real múltiples sensores ubicados en distintas áreas de producción. Cada sensor mide variables críticas como temperatura, presión y vibración de maquinaria, enviando periódicamente datos operativos a un servidor central encargado de consolidar, visualizar y registrar el estado general de la planta. Debido a la naturaleza del negocio, la solución debe diferenciar entre telemetría continua de alta frecuencia (donde se prioriza velocidad) y alertas críticas (donde se prioriza confiabilidad absoluta), simulando una arquitectura profesional similar a sistemas IoT industriales o SCADA de mercado.

El objetivo del trabajo es diseñar e implementar una solución cliente-servidor utilizando sockets, donde múltiples clientes (sensores) reporten telemetría normal mediante UDP hacia un servidor central, mientras que cualquier condición crítica (por ejemplo, temperatura excesiva o vibración anormal) genere automáticamente una alerta enviada mediante TCP para garantizar su recepción. El sistema deberá permitir monitorear varios sensores simultáneamente, registrar eventos, gestionar conexiones y analizar técnicamente por qué se selecciona cada protocolo según el tipo de comunicación.

Ejercicio a resolver

Desarrollar un sistema distribuido compuesto por múltiples programas cliente que simulen sensores industriales y un servidor central de monitoreo. Cada sensor deberá enviar periódicamente sus mediciones de temperatura, presión y vibración utilizando UDP. Cuando alguno de estos valores supere umbrales críticos predefinidos, el sensor deberá establecer una conexión TCP con el servidor para reportar la alerta de forma confiable. El servidor deberá recibir, procesar y mostrar tanto la telemetría continua como las alertas críticas, manteniendo un registro de eventos y permitiendo observar el comportamiento del sistema frente a múltiples sensores concurrentes. La solución deberá implementarse en Python, C# o NodeJS, pudiendo compararse posteriormente diferencias de implementación entre plataformas.

Arquitectura de la aplicación



Formato de mensajes

Para los mensajes a enviar se proponen una estructura en formato Json :

UDP:

```
{
  "sensor_id": "SENSOR_01",
  "temperature": 75,
  "pressure": 110,
  "vibration": 3.4,
  "status": "OK"
}
```

TCP:

```
{
  "type": "ALERT",
  "sensor_id": "SENSOR_01",
  "metric": "temperature",
  "value": 95,
  "priority": "HIGH"
}
```

Dashboard de monitoreo (modo consola o GUI opcional)

Para mostrar la información se propone realizar un dashboard en modo consola o GUI opcional, el dashboard debe mostrar la siguiente información:

SENSOR	TEMP	PRESION	VIBRACION	ESTADO
SENSOR_01	75	110	3.4	OK
SENSOR_02	95	160	9.2	ALERTA

Guía ejemplos de código

Sockets en Python

Aplicaciones UDP

Los ejemplos de **servidor y cliente UDP en Python** permiten observar una comunicación simple basada en datagramas, donde no existe una conexión previa entre las partes. El servidor crea un socket UDP, lo asocia al puerto 5005 y queda esperando mensajes mediante `recvfrom`, recibiendo tanto los datos como la dirección del cliente. Luego responde usando `sendto`, enviando el mensaje directamente a esa dirección. El cliente, por su parte, crea su socket UDP y envía un mensaje al servidor sin establecer una sesión formal. Este ejemplo es útil para comprender que UDP es rápido y simple, pero no garantiza entrega, orden ni confirmación automática de los mensajes.

Ejemplo de Servidor UDP en Python:

```
import socket

server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind(("localhost", 5005))
print("Servidor UDP escuchando en puerto 5005...")

while True:
    data, addr = server.recvfrom(1024)
    print(f"Mensaje de {addr}: {data.decode()}")
    server.sendto(b"Mensaje recibido", addr)
```

Ejemplo de Cliente UDP en Python:

```
import socket

client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server_addr = ("localhost", 5005)
client.sendto(b"Hola servidor UDP", server_addr)
data, _ = client.recvfrom(1024)
print("Respuesta:", data.decode())
```

Aplicaciones TCP

Los ejemplos de **servidor y cliente TCP en Python** muestran una comunicación orientada a conexión. El servidor crea un socket TCP, lo asocia al puerto 6000, queda escuchando con `listen` y acepta una conexión entrante mediante `accept`. A partir de ese momento se establece un canal confiable entre cliente y servidor, permitiendo enviar y recibir datos con `sendall` y `recv`. El cliente primero debe conectarse explícitamente al servidor mediante `connect`, y recién después puede intercambiar mensajes. Este ejemplo permite ver que TCP agrega confiabilidad, orden y control de conexión, por lo que resulta adecuado para aplicaciones donde no se

Ejemplo de Servidor TCP en Python:

```
import socket

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("localhost", 6000))
server.listen(1)
print("Servidor TCP escuchando en puerto 6000...")

conn, addr = server.accept()
print("Conexión desde:", addr)

while True:
    data = conn.recv(1024)
    if not data:
        break
    print("Recibido:", data.decode())
    conn.sendall(b"Mensaje recibido por TCP")
conn.close()
```

Ejemplo de Cliente TCP en Python:

```
import socket

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("localhost", 6000))
client.sendall(b"Hola servidor TCP")
data = client.recv(1024)
print("Respuesta:", data.decode())
client.close()
```

Guía paso a paso en Python

La guía práctica propone ejecutar primero los servidores y luego los clientes para visualizar claramente el comportamiento de cada protocolo. En UDP, el cliente puede enviar un datagrama directamente al servidor sin negociación previa, mientras que en TCP primero debe establecerse una conexión entre ambos extremos. Al

repetir la experiencia con los cuatro scripts, los estudiantes pueden comparar en forma concreta las diferencias entre una comunicación no orientada a conexión y una comunicación confiable orientada a conexión, relacionando el código con los conceptos teóricos de la capa de transporte.

1. Instalar Python 3.
2. Crear dos scripts (udp_server.py, udp_client.py).
3. Ejecutar primero el servidor, luego el cliente.
4. Repetir para TCP con (tcp_server.py, tcp_client.py).
5. Observar cómo UDP no necesita conexión y TCP sí.

Sockets en C#

Los siguientes ejemplos muestran cómo implementar comunicación mediante **UDP** en **.NET Core / .NET 8+**, utilizando `UdpClient` dentro de aplicaciones de consola modernas. El objetivo es que el estudiante comprenda cómo construir soluciones multiplataforma usando C# sobre infraestructura actual, compatible con Windows, Linux y macOS. UDP permite enviar mensajes sin establecer conexión previa, lo que lo hace ideal para telemetría, streaming o sistemas donde la velocidad es prioritaria.

Ejemplo de Servidor UDP en .NET Core (Program.cs)

```
using System.Net;
using System.Net.Sockets;
using System.Text;

Console.WriteLine("Servidor UDP escuchando en puerto 5005...");

using UdpClient server = new UdpClient(5005);
IPEndPoint remoteEP = new IPEndPoint(IPAddress.Any, 0);

while (true)
{
    byte[] data = server.Receive(ref remoteEP);
    string message = Encoding.UTF8.GetString(data);

    Console.WriteLine($"Mensaje de {remoteEP}: {message}");

    byte[] response = Encoding.UTF8.GetBytes("Mensaje recibido");
    server.Send(response, response.Length, remoteEP);
}
```

Ejemplo de Cliente UDP en .NET Core (Program.cs)

```
using System.Net;
using System.Net.Sockets;
using System.Text;

using UdpClient client = new UdpClient();

byte[] msg = Encoding.UTF8.GetBytes("Hola servidor UDP");
await client.SendAsync(msg, msg.Length, "localhost", 5005);
```

```
UdpReceiveResult result = await client.ReceiveAsync();

Console.WriteLine("Respuesta: " + Encoding.UTF8.GetString(result.Buffer));
```

Aplicaciones TCP en .NET Core (C#)

Estos ejemplos muestran cómo implementar un modelo **cliente-servidor TCP** utilizando `TcpListener` y `TcpClient` en .NET Core. A diferencia de UDP, TCP requiere establecer una conexión previa, pero garantiza entrega, orden y confiabilidad, siendo ampliamente utilizado en APIs, sistemas bancarios, aplicaciones empresariales y servicios backend.

Ejemplo de Servidor TCP en .NET Core (Program.cs)

```
using System.Net;
using System.Net.Sockets;
using System.Text;

TcpListener server = new TcpListener(IPAddress.Any, 6000);

server.Start();

Console.WriteLine("Servidor TCP escuchando en puerto 6000...");

using TcpClient client = await server.AcceptTcpClientAsync();

Console.WriteLine("Cliente conectado.");

using NetworkStream stream = client.GetStream();

byte[] buffer = new byte[1024];

while (true)
{
    int bytesRead = await stream.ReadAsync(buffer, 0, buffer.Length);

    if (bytesRead == 0)
        break;

    string message = Encoding.UTF8.GetString(buffer, 0, bytesRead);

    Console.WriteLine("Recibido: " + message);

    byte[] response = Encoding.UTF8.GetBytes("Mensaje recibido por TCP");

    await stream.WriteAsync(response, 0, response.Length);
}

server.Stop();
```

Ejemplo de Cliente TCP en .NET Core (Program.cs)

```
using System.Net.Sockets;
using System.Text;

using TcpClient client = new TcpClient();

await client.ConnectAsync("localhost", 6000);
```

```

using NetworkStream stream = client.GetStream();

byte[] msg = Encoding.UTF8.GetBytes("Hola servidor TCP");

await stream.WriteAsync(msg, 0, msg.Length);

byte[] buffer = new byte[1024];

int bytesRead = await stream.ReadAsync(buffer, 0, buffer.Length);

Console.WriteLine("Respuesta: " + Encoding.UTF8.GetString(buffer, 0, bytesRead));

```

Sockets en NodeJS

Ejemplo de Servidor UDP:

```

const dgram = require('dgram');
const server = dgram.createSocket('udp4');

server.on('message', (msg, rinfo) => {
  console.log(`Mensaje de ${rinfo.address}:${rinfo.port}: ${msg}`);
  server.send("Mensaje recibido", rinfo.port, rinfo.address);
});

server.bind(5005, () => console.log("Servidor UDP en puerto 5005"));

```

Ejemplo de Cliente UDP:

```

const dgram = require('dgram');
const client = dgram.createSocket('udp4');

client.send("Hola servidor UDP", 5005, 'localhost', err => {
  if (err) console.error(err);
});

client.on('message', (msg) => {
  console.log("Respuesta:", msg.toString());
  client.close();
});

```

Ejemplo de Servidor TCP:

```

const net = require('net');

const server = net.createServer(socket => {

```

```
console.log('Cliente conectado');
socket.on('data', data => {
  console.log("Recibido:", data.toString());
  socket.write("Mensaje recibido por TCP");
});
});

server.listen(6000, () => console.log("Servidor TCP en puerto 6000"));
```

Ejemplo de Cliente TCP:

```
const net = require('net');

const client = net.createConnection({ port: 6000 }, () => {
  client.write("Hola servidor TCP");
});

client.on('data', data => {
  console.log("Respuesta:", data.toString());
  client.end();
});
```